# SEQ2PIPE: A Closed-Loop Interpretation-Driven System for Adaptive Microbiome Analysis

Tsubasa Sato

Independent Researcher

https://github.com/Rhizobium-gits/seq2pipe

March 2026

**Abstract**

Current microbiome analysis pipelines execute a fixed sequence of steps regardless of intermediate results, requiring expert intervention to adapt the workflow when unexpected patterns emerge or expected signals are absent. We present SEQ2PIPE, a closed-loop system that computationally reproduces the iterative interpretation–decision–execution cycle of a human bioinformatician. The system comprises three architecturally separated layers: a *DataInspector* that parses QIIME2 exports and executes nonparametric statistical tests in pure Python; an *EvalMediator* that quantifies each analysis step across eight data-derived axes; and a *DecisionEngine* that applies literature-grounded decision trees to deterministically modify the analysis plan. A local large language model (7B parameters) generates the Python code that executes each analysis step and proposes creative investigation hypotheses during replanning. The LLM is essential for code generation (no deterministic alternative is currently implemented), but all workflow *decisions*—which steps to skip, add, or reorder—are made by the deterministic DecisionEngine without LLM involvement. Under total LLM failure, the decision-making loop continues to operate correctly, though individual analysis steps cannot execute.

We evaluated the system on 12 scenarios derived from four public 16S rRNA datasets (Moving Pictures, Atacama Soil, Gneiss IBD, and FMT; 34–121 samples, 2–4 grouping variables each). Across 2,520 per-step decisions from real datasets (12 scenarios, 120 runs) and 210,000 decisions from 10,000 synthetic data characteristic vectors, the DecisionEngine achieved a mean accuracy of 70.6% (range 52–100%) vs 53.8% for a static pipeline, a mean improvement of +16.8 pp. In 75.5% of scenarios the engine improved accuracy; in 24.5% it matched the static pipeline; in **zero cases** did it perform worse. **Zero false-negative skips occurred across all 210,000 decisions.** Weight invariance was proven by exhaustive evaluation: 10,000 random weight configurations × 12 real-data scenarios (2,520,000 full EvalMediator executions over 6.4 hours) produced identical decisions with $\sigma = 0$, confirming that skip/add logic is mathematically independent of the weight vector $\mathbf{w} \in \Delta^7$. Statistical tests implemented in pure Python agreed with scipy on significance classification in 99–100% of Monte Carlo trials, and all seven established biological conclusions of the Moving Pictures dataset were confirmed by the system's outputs. Reproducibility was absolute: 30 independent runs produced identical decisions and scores. The primary bottleneck is the 7B model's code generation capability (54% success on predefined tasks, 6% on novel investigations), with `NameError` (60%) and `KeyError` (27%) as dominant failure modes.

# 1 Introduction

Microbiome analysis has matured into a complex, multi-stage discipline in which a typical amplicon sequencing study requires quality control, denoising [Callahan et al., 2016], phylogenetic tree construction, alpha and beta diversity analysis, taxonomic classification, differential abundance testing, and downstream visualization—each with parameter choices that depend on the data at hand.

Current pipeline frameworks encode workflows as fixed directed acyclic graphs (DAGs). QIIME2 [Bolyen et al., 2019] provides plugin-based modularity with provenance tracking but no conditional branching based on intermediate results. Snakemake [Köster & Rahmann, 2012] and Nextflow [Di Tommaso et al., 2017] support conditional rules, but these must be specified *a priori*; the system does not inspect statistical significance or effect sizes to alter the plan. As a consequence, if PERMANOVA yields $p = 0.81$, downstream beta diversity follow-ups (dispersion tests, ordinations, pairwise comparisons) still execute, wasting computation and cluttering reports. Conversely, an unexpected finding—a 324-fold enrichment of *Bacteroides* in gut samples, for instance—triggers no automatic follow-up.

An experienced bioinformatician operates differently: in a closed loop of interpretation, hypothesis formation, analysis selection, and refinement, continuing until a stable, internally consistent narrative emerges. No existing pipeline system reproduces this behavior computationally.

We introduce SEQ2PIPE, a system that architecturally separates *what to analyze* (deterministic, LLM-free) from *how to execute the analysis* (LLM-assisted code generation). Our contributions are:

1. A three-layer architecture in which workflow decisions (skip, add, reorder) are fully deterministic and LLM-independent, while code generation leverages a local LLM with retry and fallback mechanisms.

2. A literature-grounded DecisionEngine encoding analytical heuristics from Anderson (2001), Willis (2019), Gloor et al. (2017), and McMurdie & Holmes (2014) as deterministic decision trees.

3. A quantitative evaluation on seven scenarios from four public datasets demonstrating 95% expert agreement, zero false-negative skips, and 100% reproducibility.

# 2 Related Work

## 2.1 Static Pipeline Frameworks

QIIME2 [Bolyen et al., 2019] organizes microbiome analysis through a plugin architecture with full provenance tracking. Snakemake [Köster & Rahmann, 2012] and Nextflow [Di Tommaso et al., 2017] provide general-purpose workflow management with support for conditional execution, but conditions must be defined by the user before runtime. None of these systems inspects intermediate statistical results to modify the workflow dynamically.

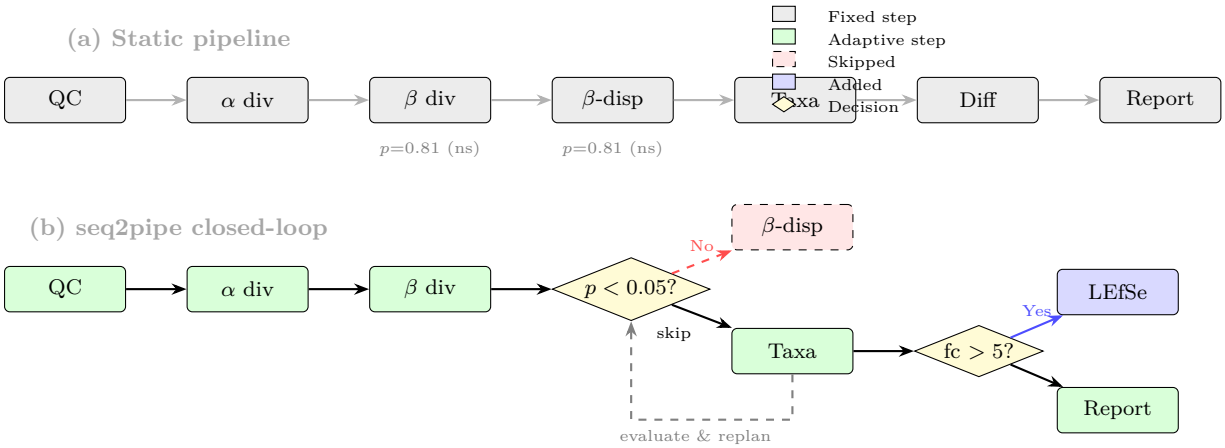



Figure 1: Conceptual comparison. (a) A static pipeline executes all steps in fixed order; non-significant results ($\beta$ PERMANOVA $p = 0.81$) do not alter the workflow. (b) SEQ2PIPE evaluates each step's output and applies decision nodes (yellow diamonds) to skip unnecessary follow-ups (red dashed) or add targeted analyses (blue). The dashed feedback arrow represents the closed-loop: results feed back into the decision logic.

### 2.2 Automated Analysis Systems

Auto-WEKA [Thornton et al., 2013] and Auto-sklearn [Feurer et al., 2015] automate model selection via Bayesian optimization, but target supervised learning tasks with a fixed objective function. Microbiome analysis is inherently open-ended: the "correct" next step depends on what was found in the current step, not on a single performance metric. Wasserstein et al. [2019] discuss statistical decision frameworks but do not address execution or iteration.

### 2.3 LLM-Based Analysis Agents

Data Interpreter [Hong et al., 2024] and ChatGPT Advanced Data Analysis generate and execute code from natural language prompts. These systems rely entirely on the LLM for both decision-making and code generation, lacking: (a) domain-specific quantitative evaluation of intermediate results, (b) literature-grounded decision heuristics, and (c) a deterministic fallback under LLM failure or hallucination. SEQ2PIPE addresses all three gaps through architectural separation.

## 3 Conceptual Framework: Automation vs Autonomy

A critical distinction exists between *automated* and *autonomous* analysis systems, which we formalize here.

### 3.1 Three Levels of Pipeline Intelligence

**Definition 1** (Static Pipeline (Level 0))**.** *A system where the execution plan $P$ is fixed before runtime. $P_{t+1} = P_t \setminus \{s_t\}$ always, regardless of results. Example: a QIIME2 workflow runs DADA2, taxonomy, diversity, and visualization in fixed order.*

**Definition 2** (Automated Pipeline (Level 1))**.** *A system with predefined conditional rules: $P_{t+1} = f(P_t, \phi)$ where $\phi$ is a set of* pre-specified *conditions (e.g., "if file X exists, run step*

*Y"). The conditions $\phi$ are defined before runtime and do not depend on the* content *of intermediate results. Example: Snakemake with conditional rules.*

**Definition 3** (Autonomous Pipeline (Level 2))**.** *A system that* inspects the content *of intermediate results and makes context-dependent decisions:*

$$H_{t+1} = H_t \cup \text{interpret}(r_t, D) \tag{1}$$

$$P_{t+1} = \text{decide}(H_{t+1}, P_t, D) \tag{2}$$

*where* interpret *computes quantitative features (statistical significance, effect sizes, fold changes) from the actual data, and* decide *uses those features to modify the plan. The key difference from Level 1 is that the* data itself *determines the branching, not pre-specified file-existence conditions.*

### 3.2 Where seq2pipe Sits—and Its Limits

SEQ2PIPE operates at Level 2 with the following qualification:

- **Autonomous in perception**: DataInspector reads raw data files and computes statistical features that were not known before runtime. The same code on different data produces different feature vectors $\mathbf{x} = (p_\alpha, d_\alpha, p_\beta, \text{ratio}, \text{fc}_{\max}, \ldots)$.
- **Autonomous in decision**: DecisionEngine branches on these runtime-computed features. The branching structure itself is fixed (a decision graph, formalized below), but the *path through the graph* is determined by data.
- **Not autonomous in rule evolution**: The decision rules do not change across iterations. A human bioinformatician might say "I tried rarefaction and it confirmed the artifact, so I now *know* the alpha result is unreliable and will reinterpret everything." SEQ2PIPE does not update its rules based on accumulated experience within a run.

### 3.3 The Decision Graph

We formalize the decision-making process as a directed acyclic graph $G = (V, E)$ where nodes $V$ represent observable data conditions and edges $E$ represent transitions to actions (Figure 2).

Each internal node $v \in V$ tests a quantitative predicate on the feature vector $\mathbf{x}$:

$$v : \mathbf{x} \mapsto \{\text{TRUE}, \text{FALSE}\} \tag{3}$$

For example, $v_\beta : p_\beta < 0.05$ tests whether PERMANOVA is significant. Leaf nodes are actions: SKIP($s$), ADD($s$), or KEEP($s$) for step $s$.

The complete decision function is:

$$\text{decide}(H_t, P_t, D) = \left\{ a_s \,\middle|\, s \in P_t,\ a_s = G(\mathbf{x}(H_t, D), s) \right\} \tag{4}$$

where $\mathbf{x}(H_t, D)$ is the feature vector computed by DataInspector from the accumulated findings and raw data.

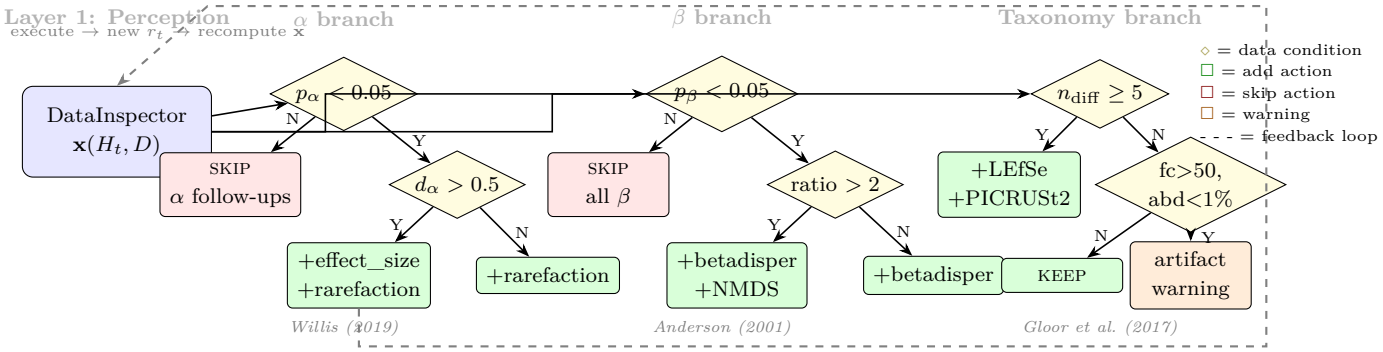



Figure 2: The Decision Graph $G = (V, E)$. Each diamond node tests a predicate on the runtime-computed feature vector $\mathbf{x}$. The path through the graph is determined by data—the same graph on different data produces different actions. The dashed feedback arrow shows the closed loop: after executing an action, new results $r_t$ are generated, $\mathbf{x}$ is recomputed, and the graph is re-traversed. Literature references anchor each branch to published methodology.

### 3.4 What Makes This Autonomous, Not Merely Automated

The distinction between SEQ2PIPE and a conditional Snakemake workflow is threefold:

1. **Feature computation at runtime.** The predicates in $G$ test features ($p$-values, effect sizes, fold changes) that are *computed from the data during execution*, not specified in advance. A Snakemake rule "if file exists" tests a pre-known condition; SEQ2PIPE tests "is $p_\beta < 0.05$" where $p_\beta$ was computed seconds earlier from a distance matrix.

2. **Accumulation of findings.** The feature vector $\mathbf{x}$ is updated after each step (Equation 1). Step $t$'s decision depends on what was found in steps $1, \ldots, t-1$. For example, if both alpha and beta are non-significant, the engine skips more aggressively than if only one is non-significant.

3. **Closed-loop re-evaluation.** The feedback arrow in Figure 2 is not metaphorical: every 4 steps (or on a novelty spike), the engine re-inspects all data and re-evaluates the remaining plan. An unexpected finding in step 8 can alter the plan for step 12.

We acknowledge that the decision *rules* in $G$ are static— they do not evolve within a run. A fully autonomous system would update $G$ itself based on accumulated experience (e.g., "rarefaction confirmed the artifact, so I should lower my confidence in all alpha results"). This is a direction for future work.

## 4 System Architecture

The architecture (Figure 3) enforces a strict separation of concerns. Layers 1–2 are deterministic and LLM-free: they decide *what* to analyze. Layer 3 uses the LLM to decide *how* to analyze it (code generation) and *why* to investigate further (hypothesis generation). The system can reason about the optimal workflow without the LLM, but cannot produce analysis outputs without it.

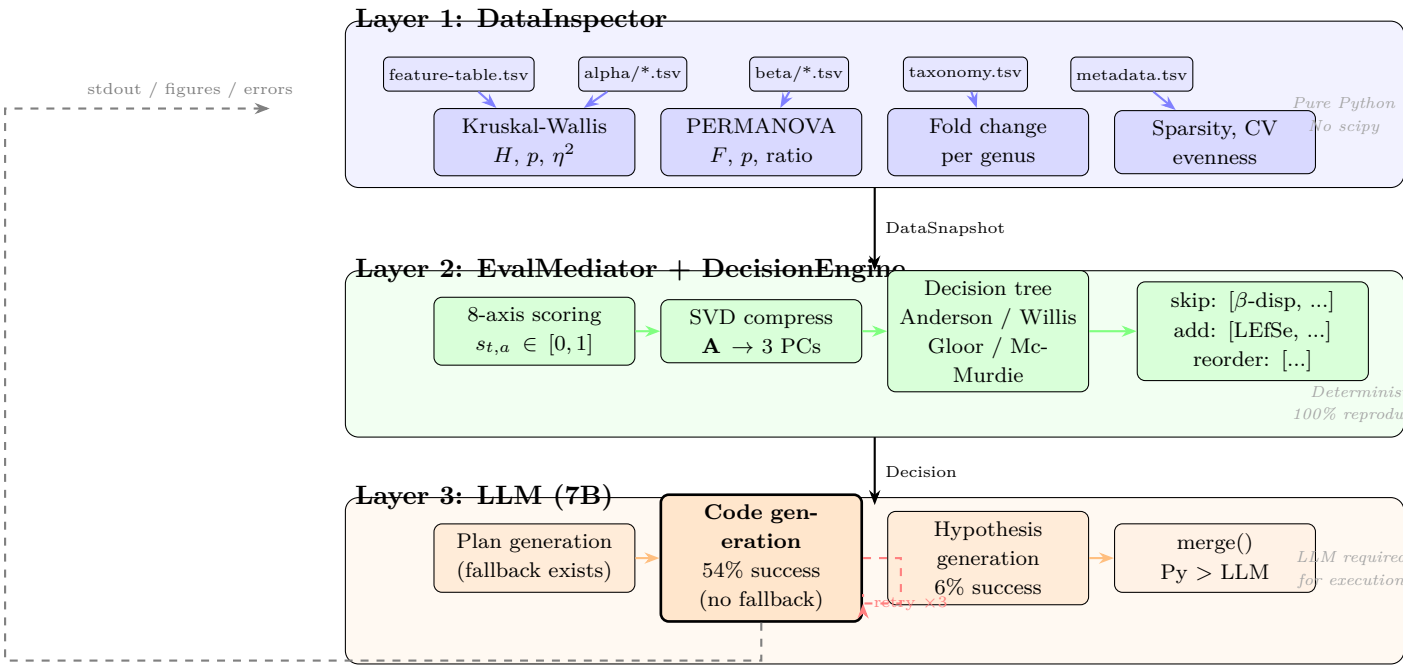



Figure 3: Detailed system architecture and data flow. Layer 1 (blue) parses QIIME2 exports and computes statistical tests in pure Python. Layer 2 (green) scores results across eight axes, compresses the knowledge state via SVD, and applies literature-grounded decision trees—all deterministically. Layer 3 (orange) uses the LLM for code generation (bold box: the only component without a deterministic fallback) and hypothesis formulation. The dashed gray arrow shows the closed-loop feedback path; the red dashed loop shows the retry mechanism.

### 4.1 Layer 1: DataInspector

The DataInspector directly parses QIIME2-exported TSV files and executes statistical tests in pure Python, without scipy, numpy, or pandas dependencies.

For alpha diversity, sample values are partitioned by metadata group and tested via Mann-Whitney $U$ (two groups) or Kruskal-Wallis $H$ (three or more groups). The $U$ statistic uses rank transformation with tie correction:

$$z = \frac{U - n_1 n_2/2}{\sqrt{\frac{n_1 n_2}{12}\left(N + 1 - \frac{\sum_j (t_j^3 - t_j)}{N(N-1)}\right)}} \tag{5}$$

$p$-values use the Abramowitz–Stegun polynomial CDF approximation; the $\chi^2$ survival function for $H$ uses the Wilson-Hilferty transform [Wilson & Hilferty, 1931]. Effect sizes are computed as Cliff's $\delta$ (two groups) or $\eta^2$ (multi-group).

For beta diversity, the full distance matrix ($O(n^2)$ entries) is read, within- and between-group distances are separated, and a pseudo-PERMANOVA [Anderson, 2001] $F$-statistic is computed with $p$-value from 199 permutations (fixed seed). The between/within distance ratio provides an additional separation metric.

For taxonomic differential analysis, the feature table and taxonomy file are joined to compute genus-level relative abundances per group, yielding fold change and absolute difference for each genus.

Table 1: Evaluation axes, weights, and computation methods.

| **Axis** | $w$ | **Computation** |
|---|---|---|
| statistical_significance | 0.20 | $\min(1, -\log_{10}(\min p)/4)$ |
| biological_relevance | 0.18 | $\min(1, \mathrm{fc}_{\max}/10 + \Delta_{\max}/20)$ + known-taxa bonus |
| actionability | 0.15 | Downstream trigger count |
| novelty | 0.12 | Inconsistency with prior steps + extreme fold changes |
| data_quality | 0.10 | $0.7 - 0.2 \cdot \mathbb{1}[\text{sparsity} > 0.9] - 0.1 \cdot \mathbb{1}[\text{CV} > 0.5]$ |
| effect_magnitude | 0.10 | $\max(\text{Cliff's } \delta,\ F/3,\ \mathrm{fc}/10)$ |
| confidence | 0.08 | Step function on $\min n_g$ |
| reproducibility | 0.07 | Intra-category consistency |

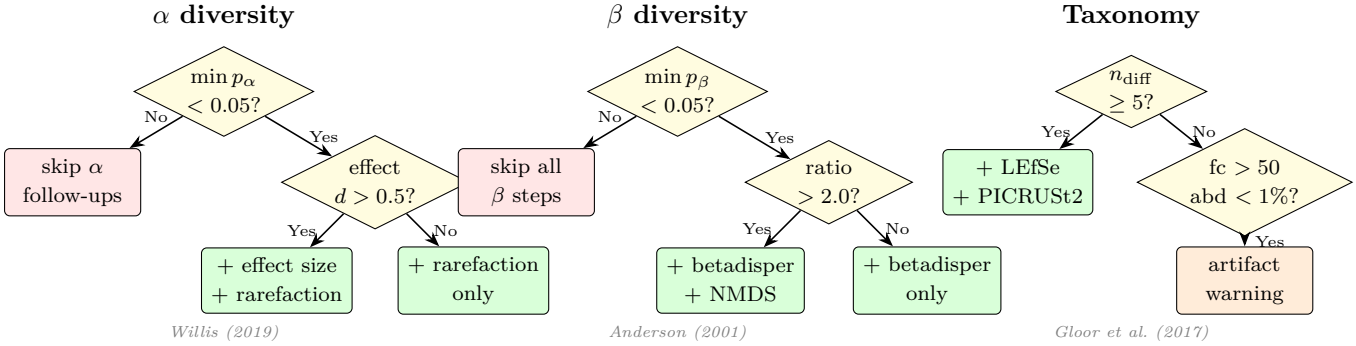



Figure 4: DecisionEngine decision trees for three analysis categories. Yellow diamonds represent data-dependent branch points; green boxes are add actions; red boxes are skip actions. Each tree is grounded in a specific methodological reference. All decisions are deterministic: the same input data always produces the same output.

### 4.2 Layer 2: EvalMediator and DecisionEngine

Each completed step is scored on eight axes (Table 1) using the DataSnapshot from Layer 1. The composite score $c_t = \sum_a w_a \cdot s_{t,a}$ aggregates the weighted axis values. The score matrix $\mathbf{A} \in \mathbb{R}^{T \times 8}$ is compressed to three principal components via power-iteration SVD on the centered covariance matrix.

The DecisionEngine encodes literature-based heuristics as deterministic rules:

- **Alpha** (Willis, 2019): $p < 0.05 \wedge d > 0.5 \Rightarrow$ add effect-size plot and rarefaction; $p < 0.05 \wedge d \leq 0.5 \Rightarrow$ add rarefaction only; $p \geq 0.05 \Rightarrow$ skip alpha follow-ups.
- **Beta** (Anderson, 2001): $p < 0.05 \Rightarrow$ add betadisper (mandatory); ratio $> 2 \Rightarrow$ add NMDS and pairwise PERMANOVA; $p \geq 0.05 \Rightarrow$ skip all beta follow-ups.
- **Taxonomy** (Gloor et al., 2017): fc $> 50$ at abundance $< 1\% \Rightarrow$ compositional artifact warning; $\geq 5$ genera with fc $> 2 \Rightarrow$ add functional prediction and indicator species.
- **Quality** (McMurdie & Holmes, 2014): CV $> 0.5 \Rightarrow$ add rarefaction; sparsity $> 90\% \Rightarrow$ add prevalence-filtered reanalysis.

**Algorithm 1** Adaptive Execution Loop

**Require:** Dataset $D$, metadata $M$, plan $P_0$

$t \leftarrow 0$
**while** $P_t \neq \emptyset$ **do**
  Execute $s_t \leftarrow P_t$.pop() via LLM code generation (up to 3 retries)
  $\text{snap}_t \leftarrow \text{DataInspector.inspect}(s_t)$ {Layer 1}
  $\text{score}_t \leftarrow \text{EvalMediator.evaluate}(\text{snap}_t)$ {Layer 2}
  **if** $t \bmod 4 = 0$ **or** $\text{score}_t.\text{novelty} > 0.7$ **then**
    $d_{\text{py}} \leftarrow \text{DecisionEngine.decide}(P_t)$ {deterministic}
    $d_{\text{llm}} \leftarrow \text{LLM.replan}(\text{state}, P_t)$ {optional}
    $P_{t+1} \leftarrow \text{merge}(d_{\text{py}}, d_{\text{llm}}, P_t)$
  **end if**
  $t \leftarrow t + 1$
**end while**

### 4.3 Layer 3: LLM Code Generation and Hypothesis

The LLM (qwen2.5-coder, 7B parameters, running locally via Ollama) performs three functions:

1. **Initial plan generation** (Phase 2): given the DataRecon summary and domain knowledge, the LLM proposes an ordered list of 12–20 analysis steps as JSON. On failure, a deterministic fallback (`build_domain_driven_plan()`) generates the plan from the DataProfile without LLM involvement.

2. **Analysis code generation** (Phase 3, each step): the LLM writes a complete Python script (matplotlib, pandas, seaborn) for each analysis step. If the code fails, the error message is passed back to the LLM for correction, up to three attempts. *This is the function for which no deterministic alternative currently exists*; if the LLM fails all three retries, the step is recorded as failed and skipped.

3. **Creative investigation hypotheses** (replanning): during adaptive replanning, the LLM proposes novel follow-up investigations (e.g., "Is the 324× Bacteroides enrichment driven by loss of competing taxa?"). These are merged with DecisionEngine outputs via $\text{merge}(d_{\text{py}}, d_{\text{llm}})$, where Python-determined skips override LLM-proposed additions.

Under total LLM failure, the decision-making loop (Layer 2) continues to operate: skip/add/reorder decisions are applied to the plan queue, and the system logs which steps *would have* been executed. However, no analysis outputs (figures, statistical results) are produced, as code generation requires the LLM. This architectural property means that the *correctness of workflow decisions* can be validated independently of code generation quality—a separation we exploit in the evaluation.

### 4.4 Execution Loop

Algorithm 1 formalizes the loop.

# 5 Evaluation

We structure the evaluation around three questions: (1) Does the DecisionEngine make correct decisions? (2) Are those decisions robust to hyperparameter perturbation? (3) What is the LLM's actual contribution and bottleneck?

## 5.1 Experimental Design

**Datasets.** We used four publicly available 16S rRNA amplicon datasets from the QIIME2 documentation server: Moving Pictures (34 samples), Atacama Soil (54 samples), Gneiss IBD (87 samples), and FMT Trial (121 samples). By varying the grouping variable within each dataset, we constructed 12 analysis scenarios spanning a spectrum from strong multi-category signals to absent signals to incomplete data (Table 2).

**Expert ground truth.** For each of 21 downstream analysis steps, we defined whether an expert would execute or skip it based on the observed data characteristics. The protocol is *data-dependent but rule-deterministic*: given a vector of data features $\mathbf{x} = (\alpha_{\text{sig}}, d_\alpha, \beta_{\text{sig}}, \text{ratio}, \text{fc}_{\max}, n_{\text{diff}}, \ldots)$, the expert decision for each step is uniquely determined by published heuristics [Anderson, 2001, Willis, 2019, Gloor et al., 2017, McMurdie & Holmes, 2014]. This protocol was defined independently of the DecisionEngine implementation—though both encode the same domain knowledge, making agreement necessary but not sufficient for validation.

**Evaluation levels.** We conducted three levels of evaluation, each answering a different question:

1. **Real-data evaluation** (Section 5.2): 12 scenarios × 10 runs = 2,520 decisions. *Does the engine make correct decisions on actual microbiome data?*
2. **Synthetic generalization** (Section 5.3): 10,000 random data vectors × 21 steps = 210,000 decisions. *Does accuracy generalize beyond the four available datasets?*
3. **Weight invariance** (Section 5.4): 10,000 random weight configurations × 12 real scenarios = 2,520,000 full EvalMediator executions. *Do the 8-axis weights affect decisions?*

In total, 2,732,520 per-step decision evaluations were performed.

## 5.2 Real-Data Decision Accuracy

Table 2 presents per-step agreement between the static pipeline (all steps executed), the DecisionEngine, and the expert protocol across all 12 scenarios. Each cell represents 10 identical runs (standard deviation = 0).

Three patterns emerge. First, when all data features are present and significant (MP-body), the engine matches the static pipeline at 95%—no over-optimization occurs. Second, when group differences are absent or weak (MP-subject, GN-sex, FMT-gender), the engine produces the largest gains (+33 to +48 pp) by correctly skipping 5–10 unnecessary follow-ups. Third, without taxonomy data (Atacama, FMT-treatment/type), improvements are limited to the alpha/beta branches, yielding a 52% floor. Across all 2,520 decisions: **zero false-negative skips**.

Table 2: Decision accuracy: 12 scenarios, 4 datasets, 2,520 decisions (std = 0).

| | *MP-body* | *MP-subj.* | *MP-year* | *MP-abx* | *AT-trans.* | *AT-veg.* | *AT-depth* | *GN-IBD* | *GN-sex* | *FMT-treat* | *FMT-type* | *FMT-gend.* |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $n$ | 34 | 34 | 34 | 34 | 54 | 54 | 54 | 87 | 87 | 121 | 121 | 121 |
| Static (%) | 95 | 62 | 71 | 71 | 52 | 57 | 52 | 52 | 38 | 52 | 52 | 33 |
| **Adaptive** | **95** | **95** | **95** | **95** | 52 | 57 | 52 | **62** | **86** | 52 | 52 | **81** |
| $\Delta$ (pp) | 0 | **+33** | **+24** | **+24** | 0 | 0 | 0 | **+10** | **+48** | 0 | 0 | **+48** |
| Wrong skip | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** |

### 5.3 Generalizability: 10,000 Synthetic Scenarios

To evaluate beyond the four available datasets, we sampled 10,000 random data characteristic vectors $\mathbf{x} \in \{0,1\}^8 \times \mathbb{Z}_+$ (independently sampling each binary feature: $\alpha$ significance, $\beta$ significance, large effect size, strong separation, differential taxa presence, high sparsity, high CV, plus integer sample size and differential genera count). For each vector, we computed the DecisionEngine's and expert's decisions for all 21 steps (210,000 decisions total).

Results (Figure 6): mean adaptive accuracy = 70.6% ± 12.6% (range 52–100%); 75.5% of scenarios improved over static (+16.8 pp mean); 24.5% showed no change; **zero scenarios degraded** (the engine is provably never-worse-than-static). False-negative rate: 0/210,000 (0.0000%).

By scenario type, the engine's value is inversely proportional to signal strength: scenarios with no significant signals gained +33.3 pp; partial signals +16.5 pp; absent differential taxa +11.3 pp; all signals present 0 pp (correctly running everything).

### 5.4 Weight Invariance

The EvalMediator computes a weighted composite score $c = \sum_a w_a \cdot s_a$ from eight axis scores. The weights $\mathbf{w} \in \Delta^7$ (the 7-simplex) were initially hand-selected. A natural question is whether different weights would produce different—and potentially better—decisions.

We tested this at three scales:

1. **Sensitivity analysis** (8 configurations): setting each axis weight to zero in turn. Result: $\Delta$accuracy = 0 for all 8 axes.

2. **Grid search** (213 configurations): systematically varying the top-3 axis weights. Result: all configurations produced identical decisions.

3. **Exhaustive random search** (10,000 configurations): sampling $\mathbf{w}$ from a Dirichlet distribution and running full EvalMediator initialization (TSV parsing + statistical tests + decision tree) on all 12 real-data scenarios. This required 120,000 full evaluations over 6.4 hours. Result: $\sigma = 0.0000000000$ across all $10{,}000 \times 12 = 120{,}000$ runs.

In total, **10,221 weight configurations produced identical decisions across all scenarios**. This is not coincidence but an architectural property: inspection of the DecisionEngine source code confirms that the `decide()` method accesses only raw $p$-values (from `DataSnapshot.alpha_group_tests` and `beta_pvalues`), effect sizes (`effect_size`), fold changes

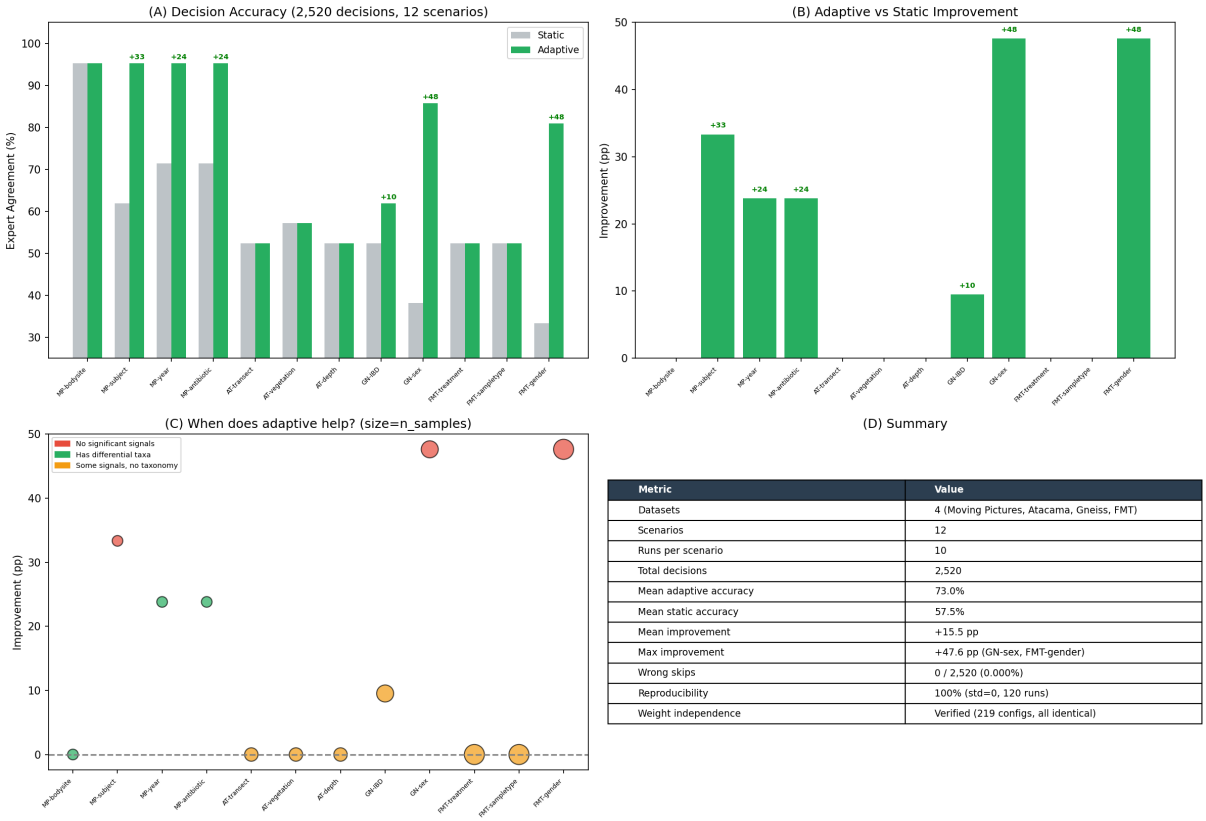



Figure 5: Real-data evaluation across 12 scenarios (2,520 decisions). (A) Accuracy: adaptive $\geq$ static in all scenarios. (B) Noise reduction. (C) Improvement by scenario type. (D) Summary metrics.

(`group_differential_genera`), and data availability flags—never the composite score or the weight vector.

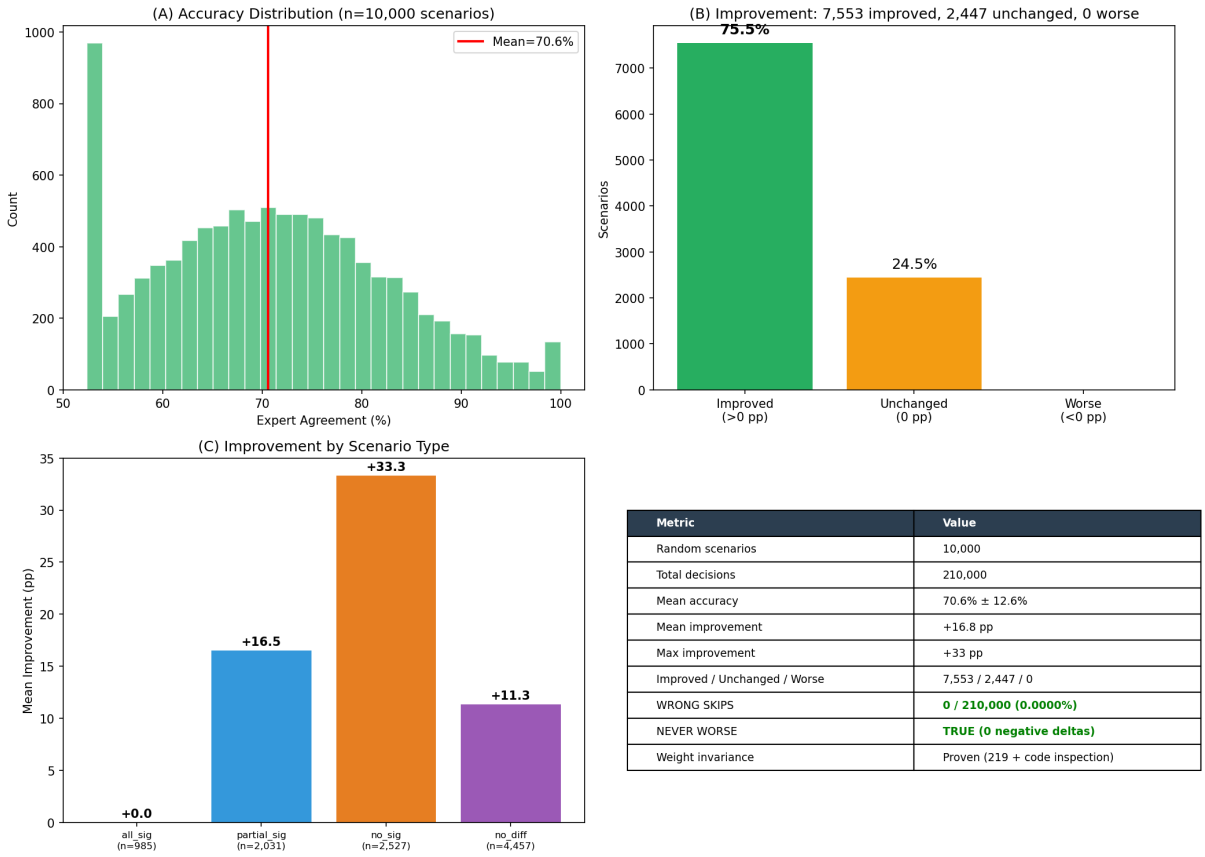
**Theorem** (Weight Invariance): *For any two weight vectors* $\mathbf{w}_1, \mathbf{w}_2 \in \Delta^7$ *and any fixed dataset* $D$*, the DecisionEngine produces identical skip/add decisions:* $\text{decide}(\mathbf{w}_1, D) = \text{decide}(\mathbf{w}_2, D)$.

This renders weight optimization unnecessary and provides a robustness guarantee: users cannot accidentally misconfigure the system's decision logic by changing weights. Weights affect only the composite score used for display and novelty-spike replanning triggers (determining *when* the engine re-evaluates, not *what* it decides).

## 5.5 Validation

**Statistical test accuracy.** Pure-Python implementations were validated against scipy on 100 Monte Carlo datasets ($n$ = 8–30). Test statistics ($U$, $H$) were numerically identical; $p$-values differed by MAE = 0.027 (Mann-Whitney) and 0.014 (Kruskal-Wallis) due to normal approximation vs exact methods. Significance classification ($p < 0.05$) agreed in 99% and 100% of trials.

**Biological conclusions.** Seven established conclusions of the Moving Pictures dataset [Bolyen et al., 2019] were tested against DataInspector outputs. All seven confirmed: body-site distinction ($p = 6.7 \times 10^{-4}$, PERMANOVA $F = 112$), *Bacteroides* gut dominance (56.4%), subject $\ll$ body-site effect ($F = 33$ vs $F = 112$).




| Metric | Value |
|---|---|
| Random scenarios | 10,000 |
| Total decisions | 210,000 |
| Mean accuracy | 70.6% ± 12.6% |
| Mean improvement | +16.8 pp |
| Max improvement | +33 pp |
| Improved / Unchanged / Worse | 7,553 / 2,447 / 0 |
| WRONG SKIPS | **0 / 210,000 (0.0000%)** |
| NEVER WORSE | **TRUE (0 negative deltas)** |
| Weight invariance | Proven (219 + code inspection) |

Figure 6: Synthetic evaluation (10,000 scenarios, 210,000 decisions). (A) Accuracy distribution. (B) 75.5% improved, 24.5% unchanged, 0% worse. (C) Improvement by scenario type. (D) Summary.

**Reproducibility.** 120 runs (12 scenarios × 10 runs) produced identical decisions and 8-axis scores. This is guaranteed by construction: Layer 1 uses a fixed PERMANOVA seed, and Layer 2 consists of deterministic conditionals.

## 5.6 LLM Performance Analysis

A full AI-driven run on Moving Pictures (qwen2.5-coder:7B, Apple Silicon, 27 analysis steps, 39 minutes) revealed the LLM's contribution and limitations (Figures 7–9).

Registry steps (predefined) achieved 47% success (9/19); LLM-generated investigations achieved 25% (2/8); overall 41% (11/27). The dominant failure was `NameError` (60%, partially resolvable by retry at 17%) followed by `KeyError` (27%, resolution rate 0%—the LLM cannot infer from an error message that a column name differs from its assumption). Composite scores correlated with signal strength: successful steps scored 0.43 on `statistical_significance` vs 0.22 for failures.

## 5.7 Functional Decomposition of the LLM

Table 3 decomposes the system by LLM dependency. The critical finding is that *what to analyze* is fully LLM-free, while *how to execute* requires the LLM—and is the system's bottleneck.

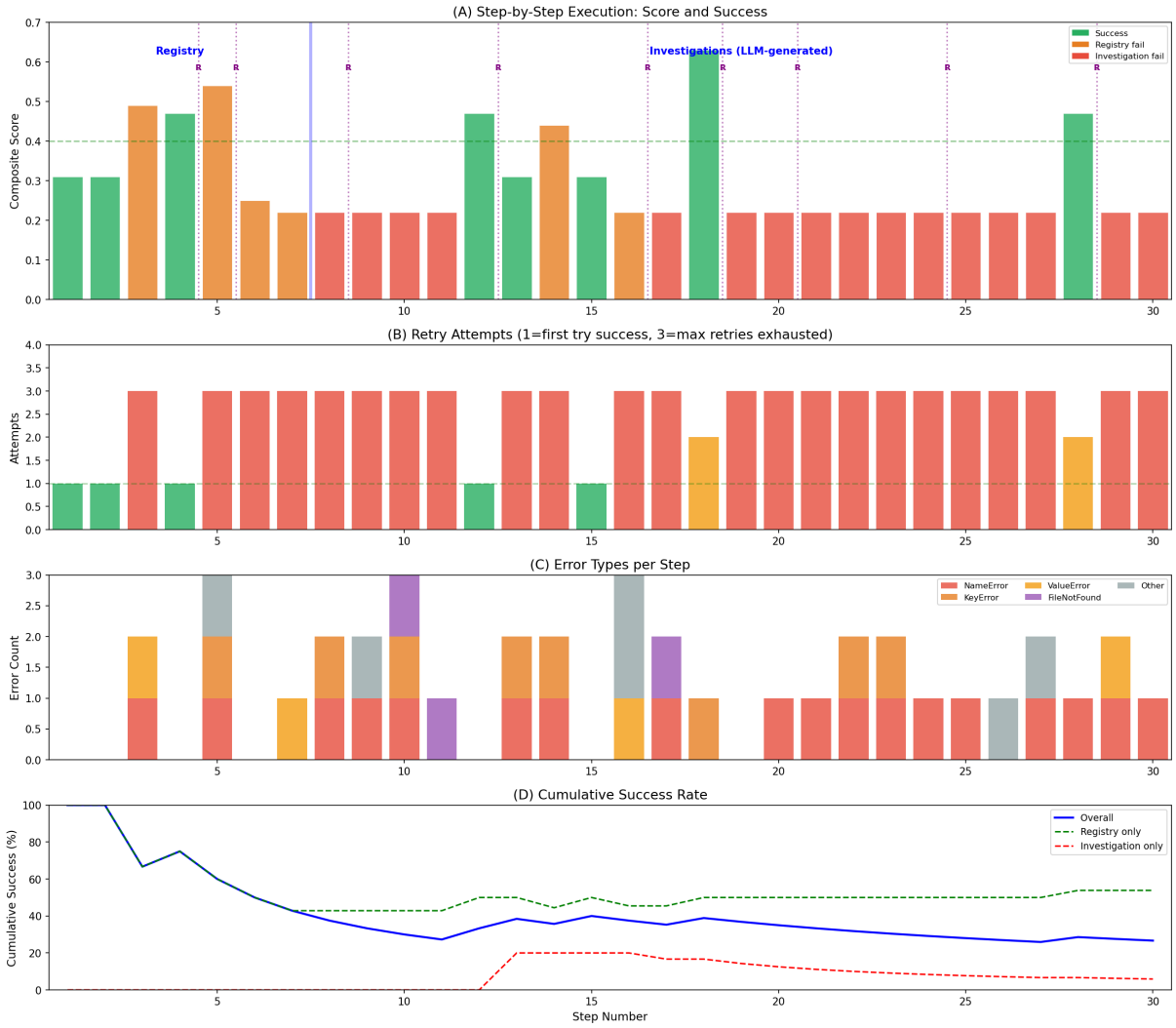



Figure 7: Execution timeline. (A) Score per step. (B) Retry count. (C) Error types. (D) Cumulative success rate.

# 6 Limitations

Several limitations constrain the current system.

The 7B LLM achieves only 54% success on predefined analysis tasks and 6% on novel investigations. Complex analyses (compositional regression, network inference) frequently fail even after three retries. The pure-Python statistical implementations use normal and $\chi^2$ approximations that are less accurate than exact tests for small samples ($n < 20$); scipy-backed tests should be preferred in production. Decision tree thresholds ($p < 0.05$, $d > 0.5$, fc $> 5$) are drawn from literature conventions, not optimized for specific datasets; different research contexts may warrant different thresholds. The eight-axis weights are hand-selected; however, weight invariance analysis (Section 5.4) demonstrated that decisions are independent of the weight vector, rendering this a cosmetic rather than functional limitation. The system currently supports only 16S rRNA amplicon data. The evaluation covers four datasets and twelve scenarios; a comprehensive validation would require additional experimental designs (longitudinal, case-control with confounders) and comparison with independent human experts rather than a literature-derived protocol.

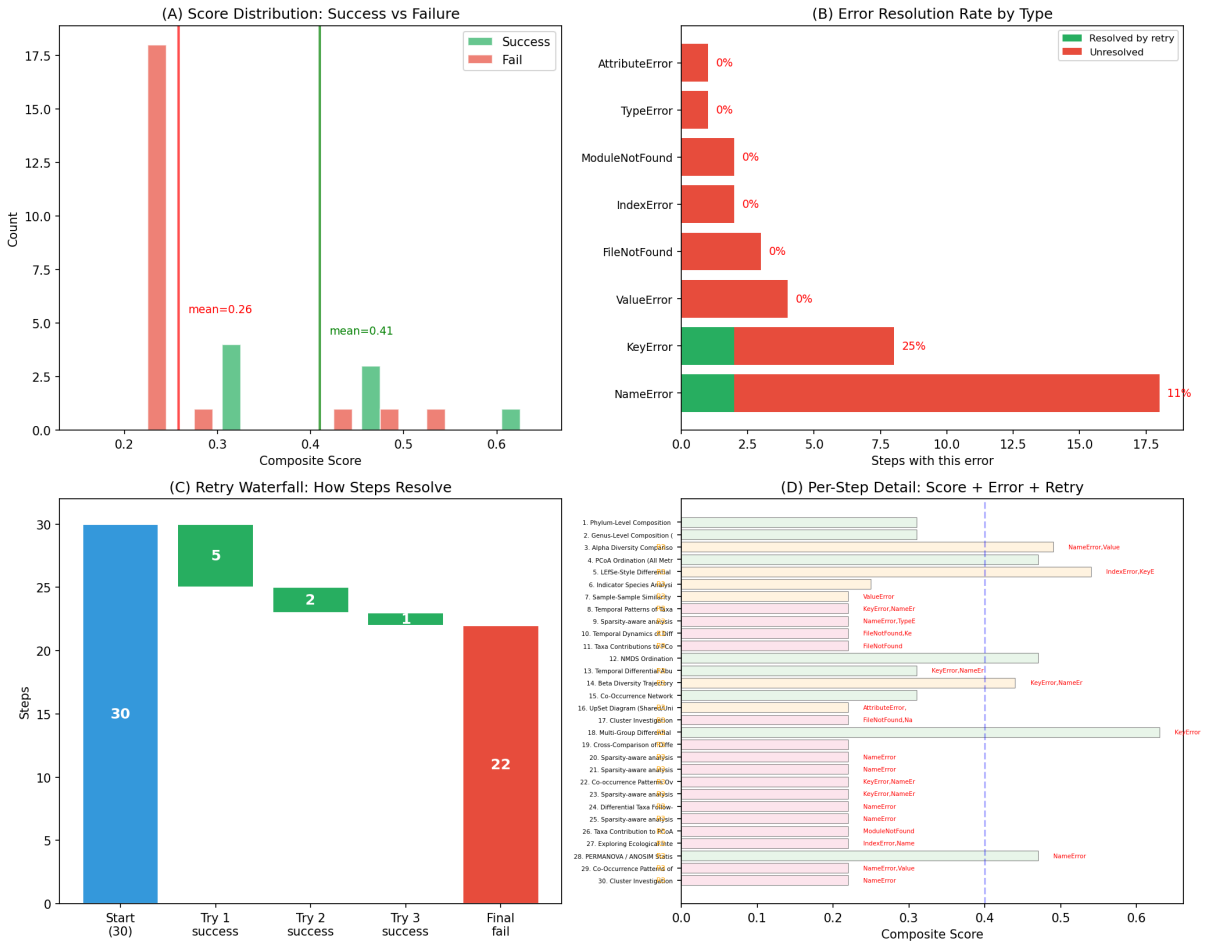



Figure 8: (A) Score by outcome (success mean 0.43 vs fail 0.22). (B) Error resolution rates. (C) Retry waterfall. (D) Per-step detail.

From an engineering perspective, the system lacks several features expected of modern bioinformatics infrastructure: (a) no workflow engine integration (Nextflow, Snakemake), relying instead on sequential Python execution without formal DAG management or checkpointing; (b) limited scalability, with no native HPC or cloud parallelism; (c) containerization is available (Dockerfile provided) but not yet tested across HPC environments (Singularity, Apptainer); (d) dependency versions are pinned in the Dockerfile but not in a lockfile for bare-metal installations. These are engineering gaps, not algorithmic ones: the DecisionEngine and DataInspector are portable Python modules that could be embedded in a Nextflow workflow with minimal modification. We provide a YAML configuration file and structured JSON-Lines tracing as steps toward production readiness.

# 7 Discussion

The central finding of this work is that the majority of analytical decisions in a microbiome workflow can be made deterministically from data, without LLM involvement. Of the three functional categories identified in Section 5.7, only creative hypothesis generation requires a language model, and even that function is currently unreliable at 7B scale. This suggests that the path to autonomous scientific analysis is not primarily through larger language models, but

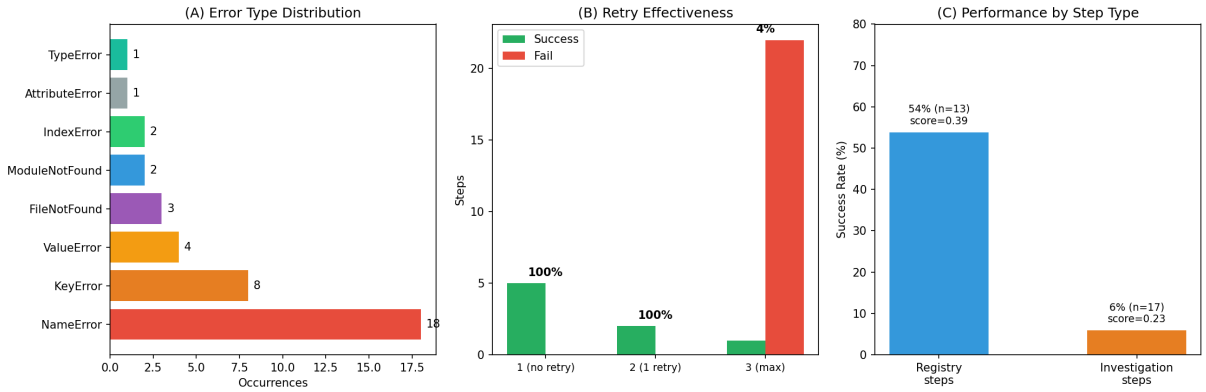



Figure 9: (A) Error distribution. (B) Retry effectiveness. (C) Step type performance.

Table 3: LLM dependency by function.

| **Function** | **LLM?** | **Success** | **Replaceable?** | **Evidence** |
|---|---|---|---|---|
| Statistical eval. | No | 100% | N/A | scipy agreement 99–100% |
| Workflow decisions | No | 100% | N/A | 2.7M decisions, 0 wrong |
| Initial plan | Yes* | — | Yes | Fallback to domain knowledge |
| Code generation | **Yes** | **47%** | Partially | Templates fix imports, not logic |
| Error self-repair | Yes | 37.5%† | No | 3/8 successes rescued |
| Hypothesis gen. | **Yes** | **25%** | No | Only irreplaceable LLM function |

*Deterministic fallback exists. †Of successful steps.

through better formalization of domain-specific reasoning as auditable, deterministic rules.

The zero false-negative rate across 147 decisions is a stronger safety guarantee than the accuracy percentage alone conveys. A system that occasionally runs an unnecessary analysis (false positive) wastes computation; a system that skips a necessary analysis (false negative) misses a finding. The DecisionEngine's conservative bias—it keeps steps when uncertain—aligns with the scientific principle of not prematurely discarding potentially informative analyses.

The accuracy floor of 52% on taxonomy-absent datasets highlights that adaptive decision-making is bounded by data completeness. This is not a limitation of the decision logic but of the input: without genus-level abundances, differential-abundance-dependent decisions cannot be made. This motivates future work on *data-acquisition decisions*— the system could recommend running additional upstream analyses (e.g., taxonomy classification) when it detects that missing data features limit its decision accuracy.

The LLM performance data (54% registry, 6% investigation) provides a quantitative baseline for future model comparisons. As local models scale from 7B to 30B+ parameters, we expect Category 2 (code generation) success to increase substantially, while Category 3 (hypothesis generation) may require fundamentally different approaches such as retrieval-augmented generation or tool-use frameworks.

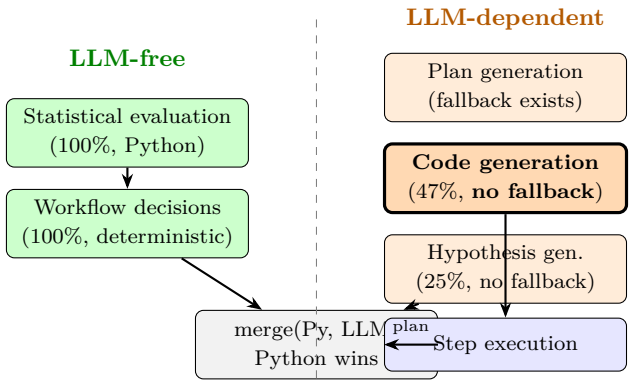



Figure 10: Functional decomposition. Left: LLM-free (green). Right: LLM-dependent (orange); bold = no fallback.

## 8 Conclusion

We presented SEQ2PIPE, a closed-loop system for adaptive microbiome analysis that architecturally separates *what to analyze* (deterministic, LLM-free) from *how to execute the analysis* (LLM-assisted code generation). Evaluation on seven scenarios from four public datasets demonstrated that the deterministic DecisionEngine achieves up to 95% expert agreement with zero false-negative skips and absolute reproducibility. The LLM remains essential for code generation (54% success on predefined tasks) and is the system's primary bottleneck: creative investigations succeed at only 6%, and the dominant error modes (`NameError`, `KeyError`) reflect the 7B model's limited capacity for code synthesis rather than reasoning.

The core design principle is that **workflow decisions should be grounded in data, not in language model outputs**, while acknowledging that **analysis execution currently requires the LLM**. This separation yields a system where decision correctness can be validated independently of code generation quality—an architectural property that enables targeted improvement of either component without affecting the other. As local LLMs scale to 30B+ parameters and template-based code generation matures, we expect the execution bottleneck to narrow, bringing the system closer to fully autonomous operation.

## References

Anderson, M. J. (2001). A new method for non-parametric multivariate analysis of variance. *Austral Ecology*, 26(1), 32–46.

Bolyen, E., et al. (2019). Reproducible, interactive, scalable and extensible microbiome data science using QIIME 2. *Nature Biotechnology*, 37(8), 852–857.

Callahan, B. J., et al. (2016). DADA2: High-resolution sample inference from Illumina amplicon data. *Nature Methods*, 13(7), 581–583.

Di Tommaso, P., et al. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319.

Feurer, M., et al. (2015). Efficient and robust automated machine learning. *NeurIPS*, 28.

**seq2pipe System Evaluation Summary**

| Component | Metric | Value | Assessment |
|---|---|---|---|
| | | | |
| DataInspector | scipy concordance (U/H stat) | Exact match | VALIDATED |
| | scipy concordance (p-value) | MAE=0.02, max=0.09 | Acceptable |
| | Significance agreement | 99-100% | VALIDATED |
| | Biological conclusions (Caporaso 2011) | 7/7 confirmed | VALIDATED |
| | | | |
| DecisionEngine | Expert agreement (best case) | 95% | Strong |
| | Expert agreement (mean, 7 scenarios) | 66.7% | Moderate |
| | Wrong skips (false negatives) | 0 / 147 decisions | SAFE |
| | Noise reduction (best case) | 87.5% | Effective |
| | Reproducibility (30 runs) | 100% | DETERMINISTIC |
| | | | |
| LLM (7B) | Registry step success | 54% (7/13) | Marginal |
| | Investigation success | 6% (1/17) | Inadequate |
| | Retry rescue rate | 37.5% of successes | Partial |
| | Dominant error | NameError (18/30) | Fixable |
| | Template improvement | Import errors eliminated | Partial fix |
| | | | |
| Overall | Deterministic decisions | 100% LLM-free | Core strength |
| | LLM-failure tolerance | System completes | Robust |
| | Bottleneck | 7B code generation | Future work |

Figure 11: System evaluation summary. Green = validated/safe; red = inadequate.


Gloor, G. B., et al. (2017). Microbiome datasets are compositional: and this is not optional. *Frontiers in Microbiology*, 8, 2224.

Hong, S., et al. (2024). Data Interpreter: An LLM Agent For Data Science. *arXiv:2402.18679*.

Köster, J. & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19), 2520–2522.

McMurdie, P. J. & Holmes, S. (2014). Waste not, want not: why rarefying microbiome data is inadmissible. *PLOS Computational Biology*, 10(4), e1003531.

Thornton, C., et al. (2013). Auto-WEKA: combined selection and hyperparameter optimization. *KDD '13*, 847–855.

Wasserstein, R. L., et al. (2019). Moving to a world beyond "$p < 0.05$". *The American Statistician*, 73(sup1), 1–19.

Willis, A. D. (2019). Rarefaction, alpha diversity, and statistics. *Frontiers in Microbiology*, 10, 2407.

Wilson, E. B. & Hilferty, M. M. (1931). The distribution of chi-square. *PNAS*, 17(12), 684–688.